Geavanceerde computerarchitecturen - Session 5

Cas Truyers en Niels Verhoeve

26 maart 2026

1 Introductie

Dit verslag bekijkt het verschil in prestatie tussen synchrone en asynchrone CUDA-functies. In voorgaande labo's was de CPU aan het wachten tot de GPU klaar was met de kernelfunctie uit te voeren, nu wordt gebruikgemaakt van CUDA streams om onafhankelijke kernels te scheiden van elkaar en te overlappen met CPU-operaties. **We ontdekken de voorwaarden waaronder de asynchrone methode winst oplevert en wanneer niet.** De volledige code is te vinden op: https://github.com/Niels-V-KU/Lab_Geavanceerde_Computerarchitectuur/tree/main.

2 Reduction Kernel

De kernel reduceert een data-array tot een enkele waarde zoals we eerder in labo 2 gezien hebben. Het kan vier bewerkingen doen: Sommatie, Product, Minimum of Maximum. We ontdekte dat de standaard uitvoeringstijd van een reductie op de GPU ($O(\log N)$) heel kort is, waardoor de GPU veel sneller klaar is dan de CPU die de nieuwe array initialiseert. **Hierdoor zagen we bij de resultaten geen verschil tussen synchrone en asynchrone uitvoering, daarom is er een 'dummy' berekening toegevoegd** (regels 17-19). Met opzet verhogen we de executietijd van de kernel, waardoor de **GPU en CPU load beter in balans** is.

```
1  __global__ void reductionKernel(float* d_in, float* d_out, int opType, int N) {
2      extern __shared__ float sdata[];
3      unsigned int tid = threadIdx.x;
4      unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
5
6      // Initiele data laden in shared memory
7      float val;
8      if (i < N) val = d_in[i]; // binnen de array: laad de echte data
9      else{
10         // (Padding)
11         if (opType == OP_PROD) val = 1.0f;
12         else if (opType == OP_MIN) val = 1e10f;
13         else if (opType == OP_MAX) val = -1e10f;
14         else val = 0.0f;
15     }
16
17     float dummy = val;
18     for(int j = 0; j < 200; j++) dummy = sinf(dummy) + cosf(dummy);
19     val = val + (dummy * 0.000001f);
20
21     sdata[tid] = val;
22     __syncthreads();
23
24     // Reductie in shared memory
25     for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
26         if (tid < s) {
27             if (opType == OP_SUM)  sdata[tid] += sdata[tid + s];
28             else if (opType == OP_PROD) sdata[tid] *= sdata[tid + s];
29             else if (opType == OP_MIN)  sdata[tid] = fminf(sdata[tid], sdata[tid + s])
    ;
30             else if (opType == OP_MAX)  sdata[tid] = fmaxf(sdata[tid], sdata[tid + s])
    ;
31         }
32         __syncthreads();
```

```
    }
    if (tid == 0) d_out[blockIdx.x] = sdata[0];
}
```

3 Synchronous

Hoewel kernel executies in CUDA van nature asynchroon zijn ten opzichte van de CPU, zorgen we er in deze code voor dat het toch synchroon gebeurd. Dit door het expliciet aanroepen van **cudaDeviceSynchronize()**, Pas nadat de kernel operatie volledig klaar is wordt de nieuwe iteratie gestart en de volgende array gevuld.

```
void synchronous(int N)
{
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    float* h_a;
    cudaMallocHost((void**)&h_a, N * sizeof(float));
    float* d_a;
    cudaMalloc(&d_a, N * sizeof(float));

    float h_result;

    for(int i = 0; i < 4; i++)
    {
        initializeArray(h_a, N);

        cudaMemcpy(d_a, h_a, N * sizeof(float), cudaMemcpyHostToDevice);

        reductionKernel<<<blocks, threadsPerBlock, threadsPerBlock * sizeof(float)>>>(
    d_a, d_a, i, N);

        cudaDeviceSynchronize();

        cudaMemcpy(&h_result, d_a, sizeof(float), cudaMemcpyDeviceToHost);
    }
    cudaFree(d_a);
    cudaFreeHost(h_a);
}
```

4 Asynchronous

Voor data asynchroon naar het device te kunnen kopiëren moeten we gebruik maken van **cudaMallocHost** in plaats van de standaard C-functie malloc. **Blijkbaar wijst standaard malloc de data toe aan pageable geheugen, waar de CUDA driver niet mee overweg kan** en dus eerst deze data naar een vaste locatie in het geheugen kopieert vooraleer hij deze naar de device (GPU) kopieert. Deze kopie van paged geheugen naar een vaste locatie gebeurd altijd synchroon, wat we willen vermijden. Dus kunnen we in plaats daarvan **cudaMallocHost** gebruiken die direct pinned (vast) geheugen gebruikt, en CUDA de data direct asynchroon naar het device kan kopiëren met de DMA.

```
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    float* h_a[4];
    float* h_result[4]; // Om resultaten apart op te vangen
    float* d_a[4];
    cudaStream_t streams[4];

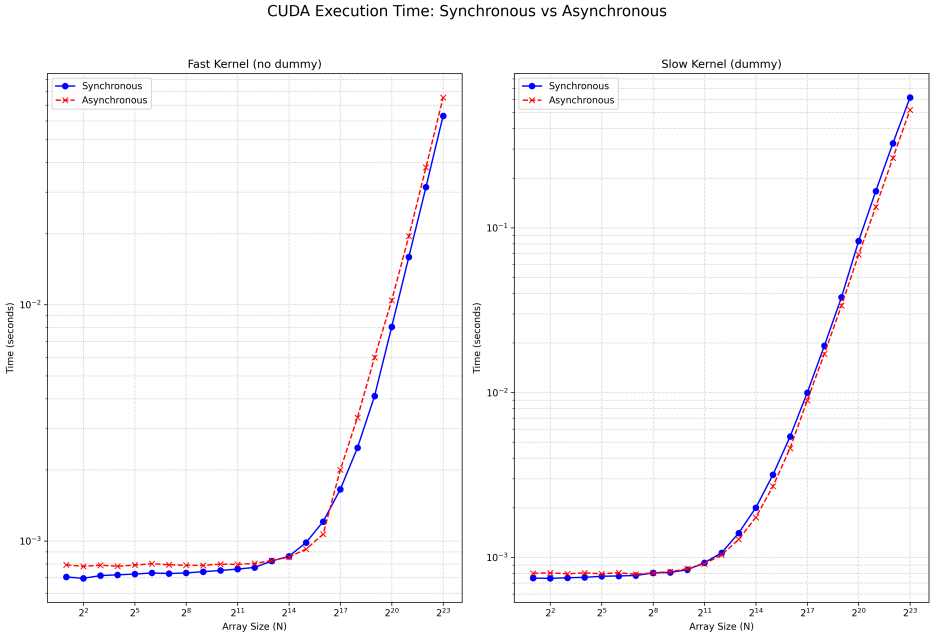
    // Allocatie
    for(int i = 0; i < 4; i++) {
        cudaMallocHost((void**)&h_a[i], N * sizeof(float));
        cudaMallocHost((void**)&h_result[i], sizeof(float));
        cudaMalloc(&d_a[i], N * sizeof(float));
        cudaStreamCreate(&streams[i]);
    }
```

```
16
17      // De ASYNC Loop: Overlap CPU voorbereiding met GPU executie
18      for(int i = 0; i < 4; i++) {
19          initializeArray(h_a[i], N);
20
21          cudaMemcpyAsync(d_a[i], h_a[i], N * sizeof(float), cudaMemcpyHostToDevice,
        streams[i]);
22
23          reductionKernel<<<blocks, threadsPerBlock, threadsPerBlock * sizeof(float),
        streams[i]>>>(d_a[i], d_a[i], i, N);
24
25          cudaMemcpyAsync(h_result[i], d_a[i], sizeof(float), cudaMemcpyDeviceToHost,
        streams[i]);
26      }
27
28      cudaDeviceSynchronize();
29      for(int i = 0; i < 4; i++) {
30          cudaStreamDestroy(streams[i]);
31          cudaFree(d_a[i]);
32          cudaFreeHost(h_a[i]);
33          cudaFreeHost(h_result[i]);
34      }
35  }
```

5 Resultaten

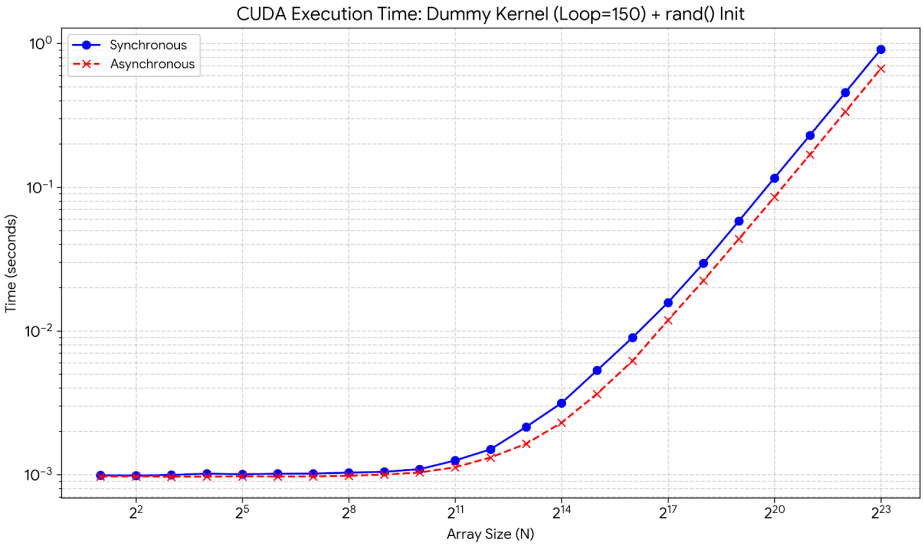
Bij de eerste tests zonder toegevoegde dummy berekening was de synchrone uitvoering identiek aan of zelfs een klein beetje sneller dan de asynchrone uitvoering. We vermoeden dat dit veroorzaakt werd door een **CPU-bottleneck**; het vullen van de arrays met de rand() functie duurde veel langer dan de datatransfer en de GPU kernel executie. Hierdoor stond de GPU nagenoeg continu te wachten op de CPU waardoor de asynchrone voordelen verdwenen, terwijl de **scheduling overhead van de streams** een vertraging veroorzaakte.

Om asynchrone winst te realiseren is de **dummy berekening toegevoegd om dit verschil in executietijd te verkleinen**. Hierdoor ontstond de benodigde overlap waarbij de CPU reeds nieuwe data genereert terwijl de GPU nog actief rekent aan een voorgaande stream. Zoals je hieronder kunt zien is de uitvoering zonder dummy (links) sneller als het synchroon wordt uitgevoerd, dit omdat er dan geen asynchrone overhead is. Op de grafiek rechts, waar er wel een dummy is gebruikt, zien we dat de asynchroon wel sneller is.



Figuur 1: Vergelijking van de executietijden in verschillende scenario's (balans CPU/GPU werklast).

Hieronder hebben we de hoeveelheid dummy uitvoeringen aangepast en getest wanneer er ongeveer het grootste verschil is tussen synchroon en asynchroon. Bij een array size van 8.388.608 is de asynchrone uitvoering 27% of 240ms sneller. **Bij een array size van 32.768 is er 32% tijdwinst.**



Figuur 2: De asynchrone schaalvoordelen zijn duidelijk zichtbaar.

6 Conclusie

Het implementeren van asynchrone CUDA-streams geeft enkel tijdwinst wanneer er sprake is van een gebalanceerde werklast tussen de CPU (host) en de GPU (device). Of toch in dit geval waar de GPU moest wachten tot de CPU klaar was met de data voor te bereiden voor de GPU. Afhankelijk van die balans zal er meer of minder tijdwinst zijn bij gebruik van asynchrone functies.